AI SOFTWARE ENGINEERING / BLOCCO 3

Lezione 13
Agent-Native Software Delivery

Task Graphs, subagent, Git Worktrees, merge e integration strategy, handoff e verifica indipendente.

Dispensa di studio avanzata, framework-independent. Contiene materiale aggiuntivo rispetto alle slide: spiegazioni, esempi formali, contratti di lavoro, scenari di errore, domande di verifica e laboratorio progressivo.

Ambito	Descrizione
Durata	35–45 minuti di lettura guidata; laboratorio supplementare di 15–25 minuti.
Prerequisiti	Lezioni 1–12: DDD, requirements, spec, TDD, contracts, architecture, context e verification.
Principio guida	Domain → Requirements → Spec → Test → Agent → Verify.
Stato	Lezione teorica erogata; esercizio proposto, non dichiarato completato.

Nota sui fatti: il caso WorkOrder usa R1 e R2 soltanto come assunzioni approvate nel laboratorio precedente. Non vengono trasferite automaticamente ai progetti software reali. La policy di fatturabilità rimane UNKNOWN.

1. Perché serve un delivery progettato per agenti

Agent-Native Software Delivery è una denominazione didattica utilizzata qui per un processo nel quale il lavoro viene disegnato in funzione della delegabilità a più agenti, senza delegare l'autorità sul dominio. Non si tratta di uno standard universale. La differenza rispetto all'impiego isolato di un coding agent è sistemica: il processo contiene artefatti autorevoli, dipendenze, strumenti di isolamento, integrazione continua e verifica esterna all'implementatore.

Senza questo livello ingegneristico, un sistema multi-agent può produrre molte modifiche che risultano corrette localmente ma non combaciano. La metrica essenziale non è il numero di patch o di prompt completati: è la capacità di generare una nuova versione integrata, verificata e utile del software.

La pubblicazione OpenAI "Harness engineering: leveraging Codex in an agent-first world" (11 febbraio 2026) documenta una sperimentazione condotta su una codebase progettata per agenti: istruzioni navigabili, vincoli architetturali verificabili, ambienti effimeri per Worktree, osservabilità locale e meccanismi di review. Si tratta di risultati specifici di quell'organizzazione, non di un benchmark automaticamente trasferibile ad altri team.

Connessione al percorso

La Lezione 11 ha definito chi possiede il modello (Strategic DDD). La Lezione 12 ha definito le unità di consistenza e gli invarianti (Tactical DDD). La Lezione 13 ha il compito di far sì che le implementazioni delegate non distruggano quei confini durante il lavoro parallelo.

2. Task Graph: pianificazione come DAG

Un Task Graph è un grafo diretto di attività e dipendenze. Nel caso usuale viene costruito come DAG (Directed Acyclic Graph), impedendo cicli di prerequisiti. Ogni nodo produce un output osservabile. Ogni arco A → B significa che B richiede l'output di A per poter procedere in modo affidabile.

Il Task Graph consente di individuare task READY, lavori bloccati, parallelismo reale, punti di integrazione e critical path. La parallelizzazione non deve anticipare decisioni ancora aperte. Se un contratto non è approvato, il lavoro del producer e del consumer può essere pianificato ma non può presumere significati condivisi inesistenti.

ID	Output	Richiede
T0	Spec approvata, ADR, confini	-
T1	Aggregate WorkOrder	T0
T2	Schema evento v1	T0
T3	Test indipendenti	T0
T4	Publisher + outbox	T1, T2
T5	Adapter Billing	T2

ID	Output	Richiede
T6	Integrazione e verifica	T3, T4, T5

Dopo T0, T1, T2 e T3 possono procedere in parallelo se hanno sufficiente definizione. T4 è dipendente sia dall'Aggregate sia dal contratto. T6 deve valutare il codice ricomposto, non limitarsi alla somma di evidenze provenienti da branch differenti.

Critical path e throughput

Il critical path è la catena di dipendenze che determina la durata minima teorica. Un Task Graph con molti task paralleli può essere più lento di un flusso sequenziale quando coordinamento, conflitti, resource contention o review diventano costosi. Valutiamo quindi tempi al merge, rilavorazioni, regressioni e difetti emersi dopo il rilascio, non soltanto task completati.

Failure mode: nessun conflitto Git, contratto rotto

Agent A pubblica un evento con proprietà reportReference. Agent B consuma una proprietà chiamata report_id. I due agenti non modificano gli stessi file, quindi Git non segnala un merge conflict. Tuttavia esiste un conflitto semantico che emerge in esecuzione o durante un contract test. Il DAG deve includere la definizione/approvazione del contratto a monte di entrambi i componenti.

3. La struttura di un task agent-ready

Il task contract è un artefatto di delega delimitata. Non coincide con la specifica della feature: descrive la porzione di lavoro consentita e le condizioni per essere accettata. Deve includere input autorevoli versionati, dipendenze, scope consentito e proibito, criteri di accettazione, output, base commit e protocollo di escalation. Le regole non approvate non diventano vere perché il task necessita di una risposta.

```
# TASK T4 - WorkOrderCompleted Publisher
STATUS: READY
DEPENDS_ON: T1, T2
INPUTS: SPEC-WORKORDER.md, ADR-001, event schema v1
ALLOWED: Operations application/persistence adapter
FORBIDDEN: Billing policy, changing approved event schema
OUTPUT: atomic WorkOrder + Outbox record, retry-safe relay
VERIFY: integration tests, failure injection, contracts
ESCALATE: contradictions, unknown business semantics
HANDOFF: commit SHA, changed files, evidence, known risks
```

La natura verificabile di questo contratto è importante. Frasi come "scrivi codice pulito" non definiscono un criterio di accettazione misurabile. "Non importare modelli interni a Billing" è invece un vincolo che può essere verificato tramite controlli delle dipendenze, review e test architetturali.

Stati del lavoro

Un set minimale di stati può essere BLOCKED, READY, IN_PROGRESS, REVIEW, INTEGRATED, VERIFIED. Sono convenzioni da adattare, non stati universali. La distinzione significativa è tra modifica prodotta, modifica integrata e comportamento verificato. I task downstream non devono essere sbloccati da una mera dichiarazione di completamento dell'implementatore.

4. Subagent, orchestrator e proprietà del contesto

Un subagent lavora su un obiettivo specifico, spesso con contesto separato e strumenti o permessi differenti. L'orchestrator mantiene la visione del grafo e il registro delle decisioni; il worker esegue una modifica o un'indagine; il reviewer produce evidenze indipendenti; l'integrator verifica la compatibilità dell'insieme. Sono ruoli logici: non è indispensabile uno specifico agente per ciascuno.

In molti casi è efficiente delegare indagini indipendenti, contract testing o code review, mantenendo nello stesso processo i passi brevi, interdipendenti o privi di una delimitazione affidabile. La documentazione OpenAI Multi-agent e Claude Code Subagents descrive contesti di lavoro separati e coordinamento, ma la scelta ottimale dipende dalla natura del task, dai costi operativi e dalla possibilità di verificare gli output.

Context budget e handoff

L'isolamento del contesto riduce la contaminazione del prompt principale con log o file non necessari. Non dobbiamo però comprimere indiscriminatamente la conoscenza: ogni subagent deve preservare riferimenti a specifiche, commit, prove, supposizioni e limiti. L'orchestrator deve distinguere sempre una conclusione supportata da evidenze da un'ipotesi plausibile.

5. Git Worktrees: isolamento delle working directory

Git Worktree permette a un repository di avere più directory di checkout, spesso associate a branch differenti. La soluzione è utile quando due agenti modificano contemporaneamente file nello stesso repository, poiché ciascuno lavora su una directory separata. Tutti i Worktree rimangono collegati al medesimo repository Git, quindi non costituiscono sandbox di sicurezza.

```
# Assumendo branch main locale aggiornato e working tree pulito
git worktree add -b feat/workorder-domain ../wt-domain main
git worktree add -b feat/workorder-contract ../wt-contract main
git worktree list

# Dopo verifica che tutte le modifiche siano salvate:
git worktree remove ../wt-domain
# Solo se il branch è già integrato e non più necessario:
git branch -d feat/workorder-domain
```

Non utilizzare git worktree prune come sostituto abituale di remove: prune ripulisce metadati di Worktree non più presenti. Prima di eliminare un Worktree controllare file non tracciati, modifiche non committate, output di test e altri artefatti. La documentazione ufficiale Git descrive i dettagli, incluso l'uso del lock per Worktree situati su unità temporaneamente non disponibili.

Che cosa Worktree NON isola

Non isola database, porte TCP, filesystem esterni, bucket, risorse cloud, secrets, account o servizi condivisi. Due agenti possono corrompere i test reciproci se invocano lo stesso database. Per un isolamento effettivo occorre valutare container, schemi/istanze distinte, servizi mock, namespace separati, dati sintetici, credenziali a minimo privilegio e teardown ripetibile.

Risorsa	Isolamento ragionevole
File checkout	Worktree e branch dedicato.
Database	Schema o istanza per task, seed e migrazioni ripetibili.
Servizi web	Porte diverse o container con namespace.
Cloud provider	Sandbox o account di test; limitare azioni distruttive.
Segreti	Accesso minimo, niente duplicazione indiscriminata di .env.

6. Merge Strategy contro Integration Strategy

La Merge Strategy disciplina il modo in cui la storia Git incorpora le modifiche: merge commit, squash merge o rebase merge. La Integration Strategy stabilisce come rendere coerenti i contributi a livello di dipendenze, contratti, persistenza e comportamento. Non sono sinonimi. Un merge meccanicamente riuscito può produrre un'applicazione non funzionante.

Nel caso WorkOrder la strategia consigliata per il laboratorio è contract-first: la proprietà e il significato di WorkOrderCompletedV1 vengono approvati prima della realizzazione di producer e consumer. I rispettivi test devono controllare compatibilità delle strutture, ma anche errori, limiti e semantica. L'owner del contratto decide versioni e modifiche incompatibili.

Integrazione frequente e cambiamenti reversibili

Continuous Integration incoraggia modifiche piccole, integrazione frequente e build automatiche. Con agenti molto rapidi, branch a lunga durata accrescono il rischio che il codice integri contratti obsoleti o presupposti non più validi. Feature flags, branch by abstraction e tecniche expand-contract permettono di integrare parti incomplete senza rendere subito visibili nuove funzionalità.

GitHub consente branch protection, required checks e merge queue. Una merge queue può far girare verifiche sul risultato che si sta per integrare contro una base aggiornata. Tuttavia i controlli non sostituiscono l'autorità sui requisiti e non garantiscono da soli la correttezza del prodotto.

7. Il ruolo dell'Integrator

L'Integrator raccoglie i contributi prodotti nei Worktree, verifica che siano basati su versioni compatibili e controlla l'intero flusso sul codice integrato. Deve osservare i contratti, i migration path, le dipendenze architetturali, i test di concorrenza e i comportamenti in errore. Se individua una contraddizione tra due regole approvate, non può risolverla cambiando una business policy arbitrariamente: deve riaprire la decisione con il relativo owner.

Ordine di integrazione proposto

Per il laboratorio: integrare il contratto approvato; verificare il modello WorkOrder; introdurre publisher/outbox; introdurre il consumer; eseguire i contract test; eseguire test di failure injection, concurrency e end-to-end; raccogliere un report indipendente su una versione/commit identificabile.

8. Durable Handoff e decision log

L'handoff durable consente a un'altra sessione o agente di proseguire senza dover ricostruire una conversazione precedente. La sua funzione non è riassumere genericamente il lavoro: deve conservare lo stato tecnico e decisionale verificabile. Gli artefatti dovrebbero essere versionati, collegati da riferimenti e aggiornati insieme al codice quando cambiano le scelte rilevanti.

```
# HANDOFF T4
Status: REVIEW
Base commit: <sha>
Branch/PR: <reference>
Changes: <files and behavior>
Tests run: <commands, environment, results>
Approved decisions: <ADR IDs>
Known UNKNOWN: event delivery SLA; dead-letter policy
Risks: pending recovery proof
Next owner: Independent Reviewer
Next gate: integrated verification T6
```

Gli UNKNOWN devono restare tali. Nel nostro esempio la policy di idoneità alla fatturazione non è stata approvata; nessun subagent può introdurre "COMPLETED = BILLABLE". La qualità del handoff è valutata anche in base alla sua capacità di trasmettere questa assenza di autorità.

9. Verifica indipendente ed Evaluation Gates

La Lezione 10 ha introdotto la differenza tra verification e validation e il rischio di correlated failure. Perciò distinguiamo la task verification (comportamento nel perimetro di una modifica), integration verification (compatibilità del sistema ricomposto) e outcome validation (effetto utile nel business). Un test eseguito dall'implementatore può fornire evidenze, ma non deve essere l'unica base per dichiarare il risultato corretto.

Il verificatore indipendente legge prima la specifica approvata e i relativi contratti, quindi esamina il commit integrato. Per ogni criterio può produrre PASS, FAIL o INCONCLUSIVE. INCONCLUSIVE è un esito corretto quando manca una decisione o non esistono prove sufficienti: evita che UNKNOWN venga silenziosamente trattato come validato.

Gate	Evidenza minima
Local	Lint, test unitari, vincoli di scope.
Contract	Producer e consumer aderenti a evento v1.
Integration	Test su commit integrato; concorrenza; retry; outbox.
Human decision	Eventuali policy nuove, permessi critici o modifiche contrattuali.

Gate	Evidenza minima
Release	Osservabilità, rollback/compensazione e criteri di rischio.

Né le soglie numeriche delle metriche, né una specifica evaluation suite di produzione sono state definite nel percorso. Restano aperte fino a una decisione esplicita. Le verifiche devono essere proporzionate al rischio reale della funzionalità.

10. Livelli di autonomia e limiti

Il grado di autonomia può crescere progressivamente: assistenza alle modifiche, task delegati con review, orchestrazione multi-task, fino a processi con integrazione e recovery automatizzati. Ogni livello richiede strumenti leggibili agli agenti, privilegi adatti, audit, controlli sulle decisioni e recupero dagli errori. Non è necessario perseguire il livello più alto per trarre beneficio dagli agenti.

In una codebase brownfield caratterizzata da scarsa documentazione, coupling o test instabili conviene iniziare da un piccolo task e introdurre gradualmente contesto, isolation e gate. In sistemi più maturi si possono automatizzare task ripetitivi e verifiche affidabili. L’esperienza OpenAI di harness engineering è utile come caso di studio, ma va letta alla luce della specifica infrastruttura e del profilo di rischio.

11. Failure mode e anti-pattern

Failure mode	Contromisura
Swarm senza pianificazione	Task Graph, ownership, READY/BLOCKED.
Worktree come sandbox	Isolamento anche di DB, porte, credenziali e servizi.
Contratti impliciti	Contract-first e owner approvato.
Merge interpretato come verifica	Test di integrazione e outcome.
DONE senza evidenze	SHA, comandi, risultati e revisioni.
Test correlati al codice	Oracoli da spec autorevole, review indipendente.
Branch troppo lunghi	PR piccole, CI frequente, merge queue dove utile.
Handoff che cancella UNKNOWN	Registro decisioni, assunzioni e questioni aperte.

12. Laboratorio progressivo completo

Obiettivo: preparare il delivery di una vertical slice “Complete WorkOrder” e dimostrare che il lavoro multi-agent può essere isolato, integrato e verificato. Non è necessario scegliere Laravel, Django o Node.js: gli artefatti e i confini sono framework-independent.

A. Task Graph

Disegnare T0–T6, indicare owner e input autoritativi e giustificare due opportunità di parallelismo. Spiegare perché T6 non può essere chiuso prima che T4 e T5 siano integrati e che T3 disponga di una base verificabile.

B. Isolamento

Assegnare una working directory/Worktree ai due implementatori. Proporre una strategia per separare database e servizio esterno di test. Descrivere il cleanup sicuro, le risorse condivise e i permessi minimi.

C. Artefatti

```
docs/delivery/TASK-GRAPH.md
docs/delivery/TASK-T4.md
docs/delivery/INTEGRATION-PLAN.md
docs/verification/T6-INDEPENDENT-VERIFICATION.md
```

D. Inject failure

Simulare una modifica incompatibile al contratto: un agent usa report_id al posto di reportReference. Non correggere a caso entrambi i lati; individuare l’owner del contratto, definire la change request, indicare i task da bloccare e i test da ripetere.

E. Acceptance criteria

Il laboratorio è accettabile quando: (a) il parallelismo rispetta il DAG; (b) le modifiche sono isolate; (c) i contratti hanno significato condiviso; (d) sono disponibili evidenze sul commit integrato; (e) non si verificano violazioni delle regole R1/R2; (f) la policy di fatturabilità rimane UNKNOWN; (g) il verificatore può riprodurre i controlli.

Domande per l’autovalutazione

- Due agenti toccano file diversi: basta per considerarli indipendenti? No: potrebbero condividere semantica e contratti.
- Perché Git Worktree non è una sandbox di sicurezza? Perché risorse esterne, credenziali e repository sottostante possono restare condivisi.
- Qual è il momento in cui un task risulta VERIFIED? Dopo controllo dei criteri autorevoli e delle evidenze, non per dichiarazione dell’implementatore.
- Che cosa deve accadere quando la spec e il contratto sono incoerenti? Escalation all’owner, senza inventare una nuova policy.

13. Collegamenti al percorso e prossimi passi

Lezione 2: linguaggio del dominio e ownership. Lezione 3: requisiti verificabili. Lezione 4: specification agent-ready. Lezione 6: architecture boundaries. Lezione 7: contract testing. Lezione 8: durable context. Lezione 9: skill, subagent, worktree. Lezione 10: evaluation e verifiche indipendenti. Lezioni 11–12: Bounded Context e Aggregate come vincoli sostanziali del delivery.

Takeaway: più agenti non significano automaticamente maggiore produttività. La prestazione reale è la capacità di integrare e verificare modifiche coerenti con il dominio. Il percorso continua con la Lezione 14, Secure Agent Engineering: prompt injection, trust boundaries, tool permissions, sandbox, supply chain e approval gates.

14. Fonti autorevoli e documentazione

- OpenAI, Harness engineering: leveraging Codex in an agent-first world, 11 febbraio 2026. https://openai.com/index/harness-engineering/
- OpenAI API Docs, Multi-agent. https://developers.openai.com/api/docs/guides/agents-api/multi-agent
- Anthropic, Claude Code - Create custom subagents. https://code.claude.com/docs/en/sub-agents
- Anthropic, Claude Code - Common workflows. https://code.claude.com/docs/en/common-workflows
- Git Project, git-worktree manual. https://git-scm.com/docs/git-worktree
- Martin Fowler, Continuous Integration, 18 gennaio 2024. https://martinfowler.com/articles/continuousIntegration.html
- GitHub Docs, About protected branches and merge queue. https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches

Stato del percorso: Lezione 13 teorica erogata; esercizi delle lezioni precedenti e del presente laboratorio non presumibilmente completati. Rimangono aperti una codebase didattica unica, un harness specifico, una suite di eval di produzione e soglie definitive per metriche.